

EECS 182 Deep Neural Networks
Fall 2026 Gireeja Ranade

Homework 4

This homework is due on Wednesday, September 30, 2026, at 10:59PM.

1. RMS Norm

We have learned how SGD and Adam can be seen as constrained optimization problems, defined by some norm over parameter space. In this problem, we will explore how it is helpful to have suitable norms over the *output features* of a neural net layer.

- (a) The Euclidean norm is often utilized to define the "scale" of a feature vector:

$$||\mathbf{x}||_2 = \sqrt{\sum_i x_i^2}.$$

However, the *root-mean-squared* (RMS) norm:

$$||\mathbf{x}||_{RMS} = \sqrt{\frac{1}{d} \sum_i x_i^2},$$

is often more appropriate when we care about the average scales of *individual feature elements*.

How do the two norms scale with dimension d (i.e. the number of elements in x), assuming each element is sampled from a standard Gaussian distribution?

Solution: For the Euclidean norm, we have:

$$\mathbb{E} [||x||_2] = \mathbb{E} \left[\sqrt{\sum_i x_i^2} \right] \leq \sqrt{\sum_i \mathbb{E} [x_i^2]} = O(\sqrt{d}) \quad (1)$$

where the inequality is from Jensen's inequality.

The RMS norm is simply the Euclidean norm divided by $\sqrt{d}$. This means that for an d -length vector, if each element of $\mathbf{x}$ is sampled from a standard Gaussian, the expected Euclidean norm scales with $O(\sqrt{d})$, while the expected RMS norm is constant $O(1)$. Thus, the RMS norm better captures the scale of individual elements.

- (b) **For a vector space with dimension $d = 2$, draw out the set of points with distance 1 under each norm below.**

- The Euclidean norm (or 2-norm):

$$||\mathbf{x}||_2 = \sqrt{\sum_i x_i^2}.$$

- The RMS Norm:

$$||\mathbf{x}||_{RMS} = \sqrt{\frac{1}{d} \sum_i x_i^2}.$$

- The 1-Norm:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|.$$

- The Infinity Norm:

$$\|\mathbf{x}\|_\infty = \max_i |x_i|.$$

Solution:

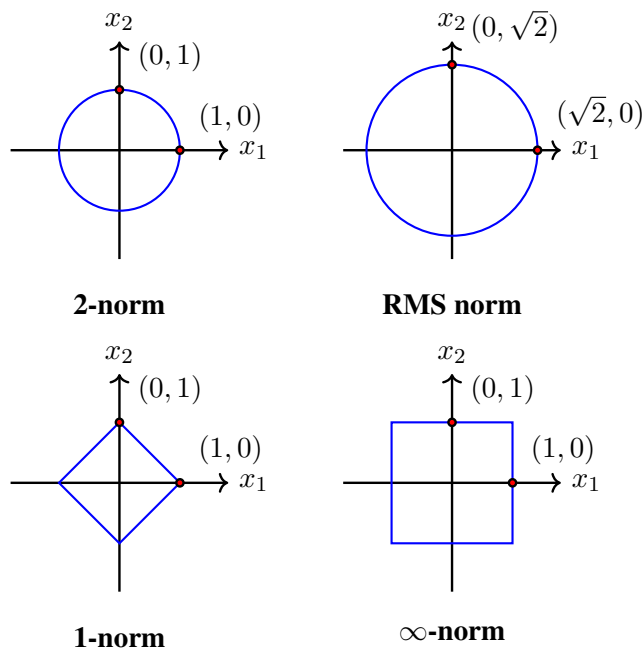


Figure 1: Unit balls for different norms in 2D. Each plot shows the set of points with distance 1 from the origin under the respective norm.

2. Optimizers as Penalized Linear Improvement with different norm penalties

In lecture, you saw the locally linear perspective of a neural network and the loss by Taylor expanding the loss around the current value of the parameters. This approximation is only very good in a near neighborhood of those values. One way to proceed with optimization is to consider the size of the neighborhood as a hyperparameter and to bound our update to stay within that neighborhood while minimizing our linear approximation to the loss. You saw in lecture that the choice of norm in defining that neighborhood also matters.

In this problem, you will work out for yourself a slightly different perspective. Instead of treating the norm as a constraint (with the size of the acceptable norm as a hyperparameter), we can do an unconstrained optimization with a weighted penalty that corresponds to the squared norm — where that weight is a hyperparameter.

At each iteration, we wish to maximize linear improvement of the objective (as defined by the dot-product between the gradient and the update) locally regularized by a penalty on the size of the update. This can be expressed (in traditional minimization form) as:

$$u = \underset{\Delta\theta}{\operatorname{argmin}} \underbrace{g^T \Delta\theta}_{\text{Linear Improvement}} + \frac{1}{\alpha} \underbrace{d(\Delta\theta)}_{\text{Distance Penalty}}, \quad (2)$$

where $g = \nabla f(\theta)$ is the gradient of the loss, α is a scalar, and d is a scalar-output distance function $\mathbb{R}^{\dim(\theta)} \rightarrow \mathbb{R}^+$.

- (a) Let's assume *Euclidean distance* is the norm that captures our sense of relevant neighborhoods in parameter space. Then our objective can be:

$$u = \underset{\Delta\theta}{\operatorname{argmin}} g^T \Delta\theta + \frac{1}{\alpha} \|\Delta\theta\|_2^2. \quad (3)$$

What is the analytical solution for u in the above problem? What standard optimizer does this recover?

Solution: To solve for u , we can take the gradient of the objective with respect to $\Delta\theta$ and set it to zero:

$$\nabla_{\Delta\theta} \left(g^T \Delta\theta + \frac{1}{\alpha} \|\Delta\theta\|_2^2 \right) = g + \frac{2}{\alpha} \Delta\theta = 0. \quad (4)$$

Then, solving for $\Delta\theta$, we get:

$$\Delta\theta = -\frac{\alpha}{2} g, \quad (5)$$

which is simply a scaled version of the negative gradient. This solution is equivalent to standard gradient descent with a step size of $\frac{\alpha}{2}$. SGD or GD can be understood as maximizing linear improvement subject to a Euclidean distance penalty over parameters.

This can also be viewed equivalently, if we split out the directions and magnitude, as:

$$\Delta\theta = -\frac{\alpha}{2} \|g\|_2 \frac{g}{\|g\|_2}, \quad (6)$$

where the $\frac{g}{\|g\|_2}$ defines the unit vector (in 2-norm) in the direction we want, and the $\frac{\alpha}{2} \|g\|_2$ tells the magnitude we want to move in that direction. Notice that this magnitude scales with the size (in 2-norm) of the gradient vector.

- (b) Now, consider an alternative way of capturing local neighborhood size – the squared infinity norm over parameters. Recall that this is defined as $\|x\|_\infty = \max_i |x_i|$. Our objective is now:

$$u = \underset{\Delta\theta}{\operatorname{argmin}} g^T \Delta\theta + \frac{1}{\alpha} \|\Delta\theta\|_\infty^2. \quad (7)$$

What is the analytical solution for u in this case? Which optimizer does this correspond to?

Solution: To solve for u , we can rewrite the objective as:

$$u = \underset{\Delta\theta}{\operatorname{argmin}} g^T \Delta\theta + \frac{1}{\alpha} \max_i |\Delta\theta_i|^2. \quad (8)$$

Let $M = \max_i |\Delta\theta_i|$. Then, we can express the objective as:

$$u = \underset{\Delta\theta, M}{\operatorname{argmin}} g^T \Delta\theta + \frac{1}{\alpha} M^2, \quad \text{subject to } |\Delta\theta_i| \leq M \text{ for all } i. \quad (9)$$

To minimize the objective, we want to make $g^T \Delta\theta$ as negative as possible while keeping M small. The optimal solution occurs when $|\Delta\theta_i| = M$ for all i , and $\Delta\theta_i$ has the opposite sign of g_i . Thus,

$$\Delta\theta_i = -M \cdot \operatorname{sign}(g_i). \quad (10)$$

From this we know the solution is the sign of the gradient, scaled by a constant $-M$. To solve for M , substituting this back into the objective, we get:

$$u = -M g^T \operatorname{sign}(g) + \frac{1}{\alpha} M^2 = -M \|g\|_1 + \frac{1}{\alpha} M^2. \quad (11)$$

To minimize this with respect to M , we set the derivative to zero:

$$\nabla_M (-M \|g\|_1 + \frac{1}{\alpha} M^2) = -\|g\|_1 + \frac{2}{\alpha} M = 0 \quad (12)$$

$$M = \frac{\alpha}{2} \|g\|_1. \quad (13)$$

Therefore, the solution for u is:

$$u_i = -\frac{\alpha}{2} \|g\|_1 \cdot \operatorname{sign}(g_i). \quad (14)$$

This is taking a step in the direction of the sign of the gradient. Intuitively, this is because the infinity norm penalizes the largest element of the update, and so the optimal solution is to make all elements of the update equal to each other in magnitude. This sign-function picks out the direction that we move to do our update. It is not the same direction as the direction of the gradient.

However, in the update above, we also have the $\|g\|_1$ scaling. This is different from SignGD. When the gradient is small, we take a small step while when the gradient is large, we take a larger step. Interestingly, we measure the size of the gradient in 1-norm while choosing the magnitude of the motion.

So both solutions have the same basic form. A magnitude that depends on the size of the gradient times a unit-vector pointing based on the gradient. However, in this case, the magnitude is measured using the 1-norm and the unit-vector is a unit-vector in the infinity-norm.

This is a manifestation of a more general pattern where the two norms involved are dual to each other. The 2-norm just happens to be its own dual.

3. Visualizing features from local linearization of neural nets

In the first discussion, you trained a 1-hidden-layer neural network with SGD and visualized how the network fitted the function leveraging the “elbows” of the non-linear activation function ReLU. In this question, we are going to visualize the effective “features” that correspond to the local linearization of this network in the neighborhood of the parameters.

We provide you with some starter code in the [course repo](#), or you can use [Google Colab](#). For this question, please submit the .pdf export of the jupyter notebook when it is completed. In addition, answer the questions below, including plots from the notebook where relevant.

- (a) **Visualize the features corresponding to $\frac{\partial}{\partial w_i^{(1)}}y(x)$ and $\frac{\partial}{\partial b_i^{(1)}}y(x)$ where $w_i^{(1)}$ are the first hidden layer's weights and the $b_i^{(1)}$ are the first hidden layer's biases.** These derivatives should be evaluated at at least both the random initialization and the final trained network. When visualizing these features, plot them as a function of the scalar input x , the same way that the notebook plots the constituent “elbow” features that are the outputs of the penultimate layer.

Solution: See notebook.

- (b) During training, we can imagine that we have a generalized linear model with a feature matrix corresponding to the linearized features corresponding to each learnable parameter. We know from our analysis of gradient descent, that the singular values and singular vectors corresponding to this feature matrix are important.

Use the SVD of this feature matrix to plot both the singular values and visualize the “principle features” that correspond to the d -dimensional singular vectors multiplied by all the features corresponding to the parameters.

(HINT: Remember that the feature matrix whose SVD you are taking has n rows where each row corresponds to one training point and d columns where each column corresponds to each of the learnable features. Meanwhile, you are going to be plotting/visualizing the “principle features” as functions of x even at places where you don't have training points.)

Solution: See notebook.

- (c) Augment the jupyter notebook to add a second hidden layer of the same size as the first hidden layer, fully connected to the first hidden layer. **Allow the visualization of the features corresponding to the parameters in both hidden layers, as well as the “principle features” and the singular values.**

Solution: See notebook.

4. Newton-Schulz Runtime

Let us consider the Newton-Schulz update for a parameter matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, using degree-3 odd polynomial p :

$$p(W) = \frac{1}{2} \left(3I_{d_{\text{out}}} - WW^T \right) W.$$

- (a) First, we will analyze the runtime of a single iteration. Assume that the runtime is dominated by matrix multiplication (which is often true for large $d_{\text{out}}, d_{\text{in}}$) and the runtime of multiplying a $n \times m$ matrix by a $m \times p$ matrix takes $cmnp$ runtime. **What is the runtime of each iteration?**

Solution: The cost consists of two matrix multiplications:

- WW^T initially is between a $d_{\text{out}} \times d_{\text{in}}$ and $d_{\text{in}} \times d_{\text{out}}$ matrix which takes $2d_{\text{out}}^2 d_{\text{in}}$ time.
- $(3I_{d_{\text{out}}} - WW^T)W$ at the end is between a $d_{\text{out}} \times d_{\text{out}}$ and $d_{\text{out}} \times d_{\text{in}}$ which takes $2d_{\text{out}}^2 d_{\text{in}}$ time.

So the total runtime is:

$$2cd_{\text{out}}^2 d_{\text{in}}.$$

- (b) Now, consider the case where $d_{\text{out}} \gg d_{\text{in}}$. **Is there a way to compute $p(W)$ faster? Explain how this can be done and report the updated runtime of each iteration?**

(Hint: Consider the Gram matrix $W^T W$ instead of WW^T . What is the runtime of computing $W^T W$? Is there a way to rewrite $p(W)$ so that it uses $W^T W$ instead of WW^T ?)

Solution: Note that we can rewrite $p(W)$ as:

$$p(W) = \frac{1}{2} (3I_{d_{\text{out}}} - WW^T) W = \frac{1}{2} W (3I_{d_{\text{in}}} - W^T W)$$

To show this, we can manipulate the original expression:

$$\begin{aligned} \frac{1}{2} (3I_{d_{\text{out}}} - WW^T) W &= \frac{1}{2} (3W - WW^T W) \\ &= \frac{1}{2} (3W - W(W^T W)) \\ &= \frac{1}{2} W (3I_{d_{\text{in}}} - W^T W) \end{aligned}$$

where we use associativity of matrix multiplication. Again, the cost of $p(W)$ consists of two matrix multiplications:

- $W^T W$ initially is between a $d_{\text{in}} \times d_{\text{out}}$ and $d_{\text{out}} \times d_{\text{in}}$ matrix which takes $2d_{\text{in}}^2 d_{\text{out}}$ time.
- $W(3I_{d_{\text{in}}} - W^T W)$ at the end is between a $d_{\text{out}} \times d_{\text{in}}$ and $d_{\text{in}} \times d_{\text{in}}$ which takes $2d_{\text{in}}^2 d_{\text{out}}$ time.

So the total runtime is:

$$2cd_{\text{in}}^2 d_{\text{out}},$$

which is better when $d_{\text{out}} \gg d_{\text{in}}$.

5. Implementing Muon (Coding Question)

In this assignment, you will implement key components of a Muon optimizer in PyTorch. For this assignment, we recommend using Google Colab as some PCs or laptops may not support GPU-based PyTorch. You will be implementing the code at [q_coding_muon.ipynb](#), then answering the questions in the notebook in your submission. The notebook itself does not need to be submitted.

- (a) Notice that in this Newton-Schulz implementation, we initially scale the Frobenius norm to be at most $\sqrt{3}$. **Can you explain why we choose this particular scaling?** Comment on why this is better than using 1.

(Hint: Newton-Schulz acts on each singular value σ of the matrix separately, sending it to $p(\sigma) = (3\sigma - \sigma^3)/2$. Find the fixed points of this map and check which of them are stable, i.e. $|p'(\sigma^*)| < 1$. What happens to a singular value that starts above the largest positive root of p ?)

Solution: We choose $\sqrt{3}$ as the positive root of the cubic polynomial. This is because for singular values $> \sqrt{3}$, Newton-Schulz (NS) using our polynomial will not converge to 1.

Note that the Frobenius norm is a conservative overestimate of the Spectral norm. When we have a mini-batch size larger than 1 and even without that in the case of a conv-net where we have weight-sharing for conv-filters across the image, we generally don't expect the gradients to be rank-1. They might be effectively low rank, but 1 is a bit extreme. Consequently, if we scale the Frobenius norm to 1, while we are being safe, we also are probably scaling many of the interesting singular values to be close to 0. Iterating the polynomial for singular values close to zero is like multiplying by $\frac{3}{2}$, the slope at the origin. This means that we expect exponential growth in those singular values as $(\frac{3}{2})^k$. Notice that $\sqrt{3} > \frac{3}{2}$ and so adjusting this scaling factor is like getting a free iteration of NS. Since these matrix multiplies in NS iterations are nontrivially expensive, this is a computational savings worth taking.

- (b) Note that Muon requires that parameters are 2D matrices of shape $d_{\text{out}} \times d_{\text{in}}$. However, we know that parameters that are convolutional kernels have shape $c_{\text{out}} \times c_{\text{in}} \times k \times k$, where c denotes number of channels and k is kernel size.

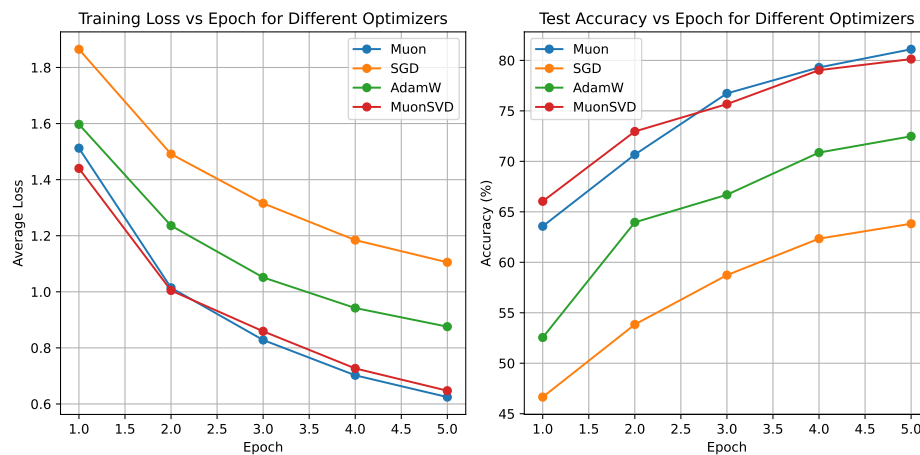
Modern implementations of convolutional layers will (implicitly) transform an input image $\mathbf{x}$ of shape $c_{in} \times h \times w$ to $\mathbf{x}'$ such that each column of $\mathbf{x}'$ has size $c_{in} \cdot k \cdot k$ and corresponds to one flattened "receptive field" of the image (or one patch of the image that a filter passes over to compute one output value). This allows the GPU to use heavily optimized matrix-multiplication capabilities directly. You may look up `im2col` for more details and take a look at <https://arxiv.org/abs/2110.03901> if you are interested.

Given this fact, **how do we modify the convolutional kernel into a $d_{out} \times d_{in}$ matrix C such that the output of the convolutional layer can be expressed as $C\mathbf{x}'$?**

Solution: Before orthogonalization, we reshape the convolutional kernel by flattening all dimensions except for c_{out} , resulting in a 2D matrix of dimensions $c_{out} \times (c_{in} \cdot k \cdot k)$. The reason why is because such flattened matrix C is the one such that $C\mathbf{x}'$ is exactly the output of the convolutional layer applied to input $\mathbf{x}$.

- (c) **Which optimizer performed best between Muon, SGD, and AdamW? Also copy the resulting plots into the submission as well.**

Solution: Here are the output plots evaluating Muon against baseline optimizers including SGD and AdamW:



It is clear from the curves that Muon performs best, achieving almost 10% higher accuracy on the test set than AdamW, and 15% than SGD. This trend is consistent across all epochs.

Even more surprising is the seeming evidence of a better inductive bias for Muon relative to SGD. Look at the training loss for Epoch 1 of Muon. It's between that of Epochs 2 and 3 of SGD. So far, this is just evidence of faster convergence in training loss. But look at the test accuracy. The test loss at Epoch 1 of Muon is better than SGD at the very end of training. This means that, at least in this phase of training (remember what you saw in the homework comparing Adam to SGD. Advantages can shift.), the mapping from training loss to test accuracy is favorable for Muon over SGD. Notice that relative to Adam, a similar tendency holds — Epoch 2 of Adam is better training loss than Epoch 1 of Muon. But the test accuracy for Muon at the end of Epoch 1 is better than Adam at the end of Epoch 2.

- (d) Compare the loss curves between Muon and MuonSVD. **Are the results expected? Explain why.**

Solution: MuonSVD performs the same as Muon. This is expected because Newton-Schulz aims to be a tractable approximation to orthogonalization, which SVD does exactly but less efficiently.

- (e) (Optional) Our implementation of Newton-Schulz is suboptimal in the polynomial used for convergence. The community has developed quintic polynomials that converge faster while still being effi-

cient. **Implement an improved Newton-Schulz and compare. Comment on the speed advantage of the improved Muon relative to the MuonSVD.**

(Hint: You can modify the number of iterations by setting the `ns_iters` parameter in the Muon optimizer)

Solution: If we replace our initial cubic polynomial with a quintic one, we are able to converge in fewer iterations (from $5 \rightarrow 3$). This makes the Muon run 25% faster than MuonSVD.

- (f) (Optional) **What were the best choices for hyperparameters for Muon? What about for AdamW?**

Solution: For Muon, the best choice for learning rate was 0.01 and for weight decay was 0.001. For AdamW, the best choice for learning rate was 0.001 and for weight decay was 0.001.

6. Maximal Update Parameterization

During lecture, we learned how SGD and Adam can be seen as constrained optimization problems, defined by some norm over parameter space. In this problem, we will explore the payoff for defining these norms over the *output features* of a dense layer and then bringing it back to the matrices involved.

- (a) Consider a dense layer with input $\mathbf{x} \in \mathbb{R}^{d_1}$, weights $W \in \mathbb{R}^{d_2 \times d_1}$, where W is initialized with i.i.d. standard Gaussian entries, and entries of $\mathbf{x}$ are sampled from an i.i.d. unit Gaussian.

What is the expected squared RMS norm of the output features $\mathbf{y} = W\mathbf{x}$? How does this scale with d_1 or d_2 ? What constant should we multiply W by to ensure that the expected squared RMS norm of $W\mathbf{x}$ is 1, regardless of d_1 and d_2 ?

Hint: Consider a simplified dense layer with a single output feature, $W \in \mathbb{R}^{1 \times d_1}$. What is the distribution of the scalar $y = W\mathbf{x}$? What is its variance?

Solution: For simplicity, first consider a single component $y_i = \sum_{j=1}^{d_{in}} W_{ij}\mathbf{x}_j$ of the output. We have that

$$\mathbb{E}[\mathbf{y}_i^2] = \text{Var}(\mathbf{y}_i) = \sum_j \text{Var}(W_{ij}\mathbf{x}_j) = \sum_i \text{Var}(W_{ij})\text{Var}(\mathbf{x}_j) = d_1.$$

The first equality holds because $\mathbf{y}$ has zero mean and subsequent equalities hold since variance is both additive and multiplicative for independent zero-mean random variables. Since this holds for each output feature, and we know that RMS norm scales with the magnitude of individual features, the RMS norm of $\mathbf{y}$ scales as $\Theta(d_1)$. Recall that multiplying a random variable by a constant c increases its variance by c^2 . To ensure that the expected squared RMS norm is 1, we should multiply W by $\frac{1}{\sqrt{d_1}}$. This recovers the Xavier initialization scaling.

- (b) We will now consider how to ensure that RMS norms of features update at a constant rate *during training*, again regardless of layer width. Assume we are using the SignGD optimizer (which is a simplified version of Adam). Unlike at initialization, where we assume weights and inputs are independent, the updates to weights during training are very much correlated with the inputs. For simplicity, assume that the minibatch training input $\mathbf{x}_i \in \mathbb{R}^{d_1}$ are sampled from an i.i.d. unit Gaussian, and the raw gradient $\nabla_W f(W)$ is an outer product of the input and a constant backpropagated vector $\mathbf{g}_i \in \mathbb{R}^{d_2}$.

$$W_{t+1} \leftarrow W_t + \eta \text{sign}(\mathbf{g}_i \mathbf{x}_i^T).$$

What is the expected RMS norm squared of the (unscaled) change in features $\mathbf{y}' = \text{sign}(\mathbf{g}_i \mathbf{x}_i^T) \mathbf{x}_i$? How does this scale with d_1 or d_2 ? What constant should we multiply the update by to ensure that the expected RMS norm of $\mathbf{y}'$ is 1, regardless of d_1 and d_2 ?

Solution: Let's consider the update to a single output feature, which in our simplified case is $y = \text{sign}(g \cdot x)^T x$, where g is a scalar constant. We can move g to the outside, resulting in $y = g \cdot$

$\text{sign}(x)^T \mathbf{x} = g \sum_{j=1}^{d_{in}} |\mathbf{x}_j|$. Note that $g^2 = 1$ and each term in the sum has a *positive* expected value that scales as $\Theta(1)$, and there are d_1 such terms, so the output feature has an expected squared RMS norm that scales as $\Theta(d_1^2)$. To ensure that the expected (squared) RMS norm is 1, we should scale the update by $\frac{1}{d_1}$. This recovers the Maximal Update Parameterization (muP) scaling.

- (c) You may notice that the above update rule only depends on d_1 . **Why is this the case?**

Solution: When we look at a weight matrix updates, the parameters that affect a single output feature are independent of the other output features. Thus, when we control for the RMS norm, we can consider each output feature independently, and the width m does not affect the scaling.

7. Homework Process and Study Group

Citing sources and collaborators are an important part of life, including being a student!

We also want to understand what resources you find helpful and how much time homework is taking, so we can change things in the future if possible.

- (a) **What sources (if any) did you use as you worked through the homework?**
- (b) **If you worked with someone on this homework, who did you work with?**
List names and student ID's. (In case of homework party, you can also just describe the group.)
- (c) **Roughly how many total hours did you work on this homework?**

Contributors:

- Kevin Frans.
- Anant Sahai.
- Matteo Guarrera.
- Mert Cemri.
- Sizhe Chen.
- Sheng Shen.
- Joey Hong.
- Gireeja Ranade.